

whisper.cpp — Three Vulnerabilities in Parakeet Model Loader

src/parakeet.cpp | Unvalidated n_fft field | Three distinct failure modes

Product	ggml-org/whisper.cpp
Version	1.9.1 — commit 5ed76e9a
Component	src/parakeet.cpp — Parakeet ASR model loader
Affected since	commit 9efddafb (2026-06-16, Parakeet support added)
Reporter	Bruce Mikel mikelbruce230@gmail.com
Disclosure	Private — June 28, 2026
Policy	90-day responsible disclosure

Summary

Three vulnerabilities in **src/parakeet.cpp** allow a crafted Parakeet model file to crash or silently corrupt any application that loads it. All three share the same root cause: **n_fft** is read from the model file at line 1013 with no range validation, and **mel_cache.init(n_fft)** is called at line 3114 outside any try/catch block. No existing CVE covers this code path. No existing fuzz harness in the project exercises the Parakeet loader. The Parakeet support was added nine days before discovery.

#	Trigger	Effect	When	CWE
1	n_fft < 0	std::terminate() — process killed	Model load	CWE-248
2	n_fft = 0	Stack overflow — process killed	First audio	CWE-674
3	n_fft prime ≥46349	OOB heap read — silent	Every frame	CWE-190/125

Root Cause

parakeet_init_with_params_no_state wraps **parakeet_model_load** in try/catch but calls **mel_cache.init()** outside the handler after the model loads:

```
// parakeet.cpp:3084-3116
try {
    model_loaded = parakeet_model_load(loader, *ctx); // n_fft read here, no check
} catch (const std::exception & e) { ... }
```

```

catch (...) { ... }

if (!model_loaded) { delete ctx; return nullptr; }

ctx->mel_cache.init(ctx->model.hparams.n_fft); // LINE 3114 -- OUTSIDE try/catch

```

`n_fft` is read from the model file with no bounds check:

```

// parakeet.cpp:1013
read_safe(loader, hparams.n_fft); // attacker-controlled int32_t, no validation

```

Vulnerability 1 — Uncaught Exception → `std::terminate()` at Model Load

Trigger

Set `n_fft` to any negative `int32_t` in the model file (e.g. `-1`).

Root Cause

`mel_cache::init` calls `std::vector::resize(n_fft)`. A negative `int32_t` converts to a value near `SIZE_MAX`, throwing **`std::length_error`**. Because the call site has no enclosing `try/catch` and the function has C-linkage, the exception propagates as undefined behavior, invoking **`std::terminate()`** on all tested platforms.

Confirmed Output

```

parakeet_model_load: n_fft = -1
parakeet_model_load: WARN no tensors loaded from model file - assuming empty model for testing
#7 std::terminate()
#9 std::__throw_length_error()
    482 sin_vals.resize(n_fft);
    3114 ctx->mel_cache.init(ctx->model.hparams.n_fft);
terminate called after throwing an instance of 'std::length_error'
Aborted (core dumped)

```

Vulnerability 2 — Infinite Recursion → Stack Overflow at Inference

Trigger

Set `n_fft` to 0.

Root Cause

`mel_cache::init(0)` succeeds — all `resize(0)` calls are no-ops. The model loads and returns a valid context. The crash fires when audio is processed. `fft()` at `parakeet.cpp:2619` has no guard for `N == 0`:

```

static void fft(float* in, int N, float* out, const parakeet_mel_cache& cache) {
    if (N == 1) { ... return; } // N=0 does not match
    const int half_N = N / 2; // 0 / 2 = 0
    float* even = in + N; // in + 0 = in (same pointer)
    fft(even, half_N, ...); // fft(in, 0, ...) -- infinite recursion
}

```

Confirmed Output

```

parakeet_model_load: n_fft = 0
WARN no tensors loaded from model file - assuming empty model for testing

```

```
parakeet_init_with_params_no_state: initialized mel cache with n_fft = 0  
[stack overflow fires on first call to any inference function]
```

Vulnerability 3 — Integer Overflow → Out-of-Bounds Heap Read

Trigger

Set `n_fft` to any prime ≥ 46349 (smallest trigger: 46349).

Root Cause

When `n_fft` is prime, `fft()` falls through to `dft()` with `N = n_fft`. Inside `dft()` at `parakeet.cpp:2597`, the lookup index overflows:

```
int idx = (k * n * sin_cos_step) % cache.n_fft; // overflow when k*n > INT_MAX  
re += in[n] * cache.cos_vals[idx]; // OOB READ when idx < 0
```

For `n_fft = 46349`, at `k = n = 46348`:

```
46348 * 46348 = 2,148,137,104 (exceeds INT_MAX = 2,147,483,647)  
            -> wraps to -2,146,830,192 on x86  
idx = -2,146,830,192 % 46349 = -37,210  
cache.cos_vals[-37210] -> reads 148,840 bytes before the buffer
```

On Linux/glibc the OOB address lands in the adjacent `sin_vals` allocation — no crash, ASan does not fire. Only UBSan catches the root cause. On other platforms the OOB may land in unmapped memory, producing a SEGV — a stronger manifestation of the same bug.

Confirmed Output (UBSan)

```
parakeet_model_load: n_fft = 46349  
WARN no tensors loaded from model file - assuming empty model for testing  
parakeet_init_with_params_no_state: initialized mel cache with n_fft = 46349  
runtime error: signed integer overflow: 46334 * 46348 cannot be represented in type 'int'
```

Reproduction Steps

Tested on: Ubuntu 24.04, clang 18.1.3, x86-64. Requirements: whisper.cpp source, clang with ASan, cmake.

Step 1 — Build whisper.cpp

```
cmake -B build -DCMAKE_C_COMPILER=clang -DCMAKE_CXX_COMPILER=clang++ \  
-DCMAKE_BUILD_TYPE=Debug -DWHISPER_BUILD_EXAMPLES=ON \  
-DGGML_NATIVE=OFF -DWHISPER_SDL2=OFF  
cmake --build build -j$(nproc)
```

Step 2 — Build the Reproduction Runner (run_poc.cpp attached)

```
clang++ -g -O0 -fsanitize=address \  
-Iinclude -Iggml/include -I. -Isrc \  
-DPARAKEET_VERSION='"test"' \  
run_poc.cpp src/parakeet.cpp \  
-Lbuild/bin -lwhisper -lggml -lggml-base -lggml-cpu \  
-Wl,-rpath,'$ORIGIN/build/bin' -lm \  
-o run_poc
```

Step 3 — Run PoC Files

```
export LD_LIBRARY_PATH=build/bin  
  
./run_poc poc_nfft_neg1.bin # Bug 1: terminate() immediately  
./run_poc poc_nfft_zero.bin # Bug 2: valid context; crash at inference  
./run_poc poc_nfft_46349.bin # Bug 3: valid context; UBSan at DFT
```

Note: The PoC files use the `n_loaded == 0` path that `parakeet_model_load` explicitly accepts as a testing shortcut. This is the minimal trigger. Any model file with a crafted `n_fft` value and valid tensor data reaches the same vulnerable `mel_cache.init` call at line 3114, since it is called unconditionally regardless of tensor count. `make_poc_final.py` (attached) documents the exact binary format.

Suggested Fix

Add one validation block after `parakeet_model_load` returns true, before `mel_cache.init` is called:

```
// parakeet.cpp -- after parakeet_model_load returns true:  
if (ctx->model.hparams.n_fft <= 0 || ctx->model.hparams.n_fft > (1 << 20)) {  
    PARAKEET_LOG_ERROR("%s: invalid n_fft: %d\n", __func__,  
                       ctx->model.hparams.n_fft);  
  
    delete ctx;  
    return nullptr;  
}  
ctx->mel_cache.init(ctx->model.hparams.n_fft);
```

Additional hardening:

```
// fft() -- add N <= 0 base case:  
if (N <= 0) return;  
  
// dft() -- use 64-bit arithmetic to prevent signed overflow:  
int64_t idx64 = ((int64_t)k * n * sin_cos_step) % (uint32_t)cache.n_fft;  
int idx = (int)idx64;
```

Novelty Confirmation

Searched NVD, CVEDetails, GitHub Security Lab, and the whisper.cpp issue tracker. No prior CVE covers src/parakeet.cpp. The file was added on 2026-06-16 and has not been audited or fuzzed. The existing stb_vorbis CVEs (CVE-2023-45681, CVE-2023-47212) cover the audio decoder path only and are unrelated.

Attachments

poc_nfft_neg1.bin	98-byte crafted model file — triggers Vulnerability 1
poc_nfft_zero.bin	98-byte crafted model file — triggers Vulnerability 2
poc_nfft_46349.bin	98-byte crafted model file — triggers Vulnerability 3
run_poc.cpp	Reproduction runner — build and run against whisper.cpp
make_poc_final.py	PoC generator — documents binary format, regenerates all three files

Reporter

Bruce Mikel

mikelbruce230@gmail.com

brucemikel.com

Associate's Degree in Cybersecurity & Networking, Magna Cum Laude — DeVry University

Independent security researcher — all testing performed on a local, isolated build. No production systems were accessed or affected.

Disclosure submitted privately on June 28, 2026. Details will not be published publicly until a patch is released, within a maximum 90-day window from maintainer acknowledgement.